AI Engineering and Claude Architect · Part I, Foundations

AI and ML Fundamentals, Lesson 4

From Word Counts to Word Vectors

A customer asks for a "refund". The policy says "reimbursement". To a word-counting system, those two share nothing at all. This lesson traces how language modeling got past that problem, and why it led straight to the transformer.

Module	AI01 of 13
Lesson	4 of 4, including the module revision
Prerequisites	AI01 L1–L3
Exam relevance	Out of scope. Sets up AI02 and AI04
Reading time	~35 minutes

Contents

1. From Word Counts to Word Vectors
2. Module Revision

Lesson 4 of 4 · AI and ML Fundamentals

From Word Counts to Word Vectors

Key idea

Language modeling advanced by finding better answers to one question: how do you turn words into numbers that preserve meaning? Counting words loses meaning and order. Learning from a word's surroundings gives vectors where similar meanings sit close together. Sequence models added order, but squeezed everything through a fixed-size memory. **That bottleneck is the problem attention was invented to solve**, which is where AI02 begins.

> *Before: neural network training and vanishing gradients (AI01 L3). Teaches: NLP basics, bag of words, TF-IDF, word vectors, RNN, LSTM, GRU, sequence-to-sequence and autoencoders. Used by: AI02 L2 (attention), AI04 L1 (embedding models) and AI04 L3 (BM25, which builds on TF-IDF). HLD M21A covers vector search over embeddings.*

The hook: the same request in different words

Your support platform routes tickets. A customer writes "refund for the order". Your knowledge base has an article titled "issue a reimbursement". Your keyword system compares them:

```
'refund for the order' vs 'issue a reimbursement': words=0.000 context=0.549
'refund for the order' vs 'courier delivery late': words=0.000 context=0.393
'refund for the order' vs 'password reset fails': words=0.000 context=0.004
```

Key idea

By word overlap, all three score exactly zero. The system can't tell the right article from an unrelated one. The `context` column comes from the method this lesson builds, and it does separate them, even on a corpus of 12 sentences.

Output is real, from `code/ai01-ml/TicketVectors.java`. The corpus is small, invented and deliberately convenient, so these values only illustrate the idea.

Intuition: you can tell a word by its neighbors

Suppose you've never seen the word tezgüino, and you read these:

→ A bottle of tezgüino is on the table.

→ Everybody likes tezgüino.

→ Tezgüino makes you drunk.

→ We make tezgüino out of corn.

You soon guess it's an alcoholic drink, though nobody defined it. This example comes from the linguistics literature, and the underlying principle is called the **distributional hypothesis**: **words that appear in similar contexts tend to have similar meanings.** The word vectors in this lesson, and the embeddings in Claude's own early layers, descend from this idea.

What NLP is

Natural language processing (NLP) is computing over human language. Language is difficult because it is:

Property	Example	The challenge
Ambiguous	"charge" a card, or "charge" a battery	One word, several meanings
Context-dependent	"It failed"	What is "it"?
Order-sensitive	"Refund denied, not approved" versus "Refund approved, not denied"	Same words, opposite meaning
Varied	"refund", "money back", "reimbursement"	Many words, one meaning
Open-ended	New product names and slang	A fixed vocabulary always falls behind

Each approach below fixed some of these rows, and exposed the next.

Stage 1: Counting words

Bag of words

Represent a text as a vector counting each vocabulary word. It's simple, fast and loses all word order, hence the name.

```
static double[] bagOfWords(String text, List<String> vocab) {
    double[] v = new double[vocab.size()];
    for (String token : tokens(text)) {
        int i = vocab.indexOf(token);
        if (i >= 0) {
            v[i]++;
        }
    }
    return v;
}
```

Compare two documents by cosine similarity, which AI02 and HLD M21A use too. It measures the angle between vectors, giving 1 for the same direction and 0 for no shared direction.

Bag of words fails for "refund" and "reimbursement":

```
bag of words: cosine(refund, reimbursement) = 0.000
```

They occupy different dimensions, so they're as unrelated as any two words. Every word is equally far from every other word.

TF-IDF

Common words like "the" dominate raw counts and carry little meaning. **TF-IDF** fixes this by weighting a term's count by how rare it is:

```
idf(term) = ln(N / documents containing the term)
```

In our 12 tickets:

Term	Tickets containing it	IDF
refund	3	`ln(12/3)` = 1.386
the	10	`ln(12/10)` = 0.182

The rare, informative word now counts more than seven times as much as "the". TF-IDF was a major practical advance. **BM25**, the ranking formula used in many search engines, refines the idea (HLD M09). You'll combine BM25 with vector search in AI04 L3, because keyword methods remain better for exact terms like `ERR-4012`. This old technique is still in production.

Naive Bayes or logistic regression over TF-IDF is still a sensible, cheap baseline for classifying tickets. Try it before anything more expensive.

Stage 2: Learning from context

Context vectors, by counting

Build a vector for each word from the words near it, using a sliding window. Raw counts favor common words, so we use **PPMI** (positive pointwise mutual information), which scores how much more often two words appear together than chance would predict:

```
double pmi = Math.log(counts[i][j] * total / (rowSums[i] * rowSums[j]));
v[j] = Math.max(0, pmi);
```

Real output:

```
context vectors: cosine(refund, reimbursement) = 1.000
context vectors: cosine(refund, approved) = 0.204
context vectors: cosine(refund, courier) = 0.000
context vectors: cosine(refund, password) = 0.008
```

> *Don't over-read that 1.000*
>
> I wrote the corpus so that "refund" and "reimbursement" appear in identical sentences, to make the mechanism visible. In real text, they'd be close, not identical. The point is that **two words that never appear together became similar, because they share neighbors.**

Word vectors, by prediction

Counted vectors are huge, one dimension per vocabulary word, and mostly zeros. word2vec (Mikolov and colleagues, 2013) learned short, dense vectors instead, with a clever trick: train a tiny network on a made-up prediction task.

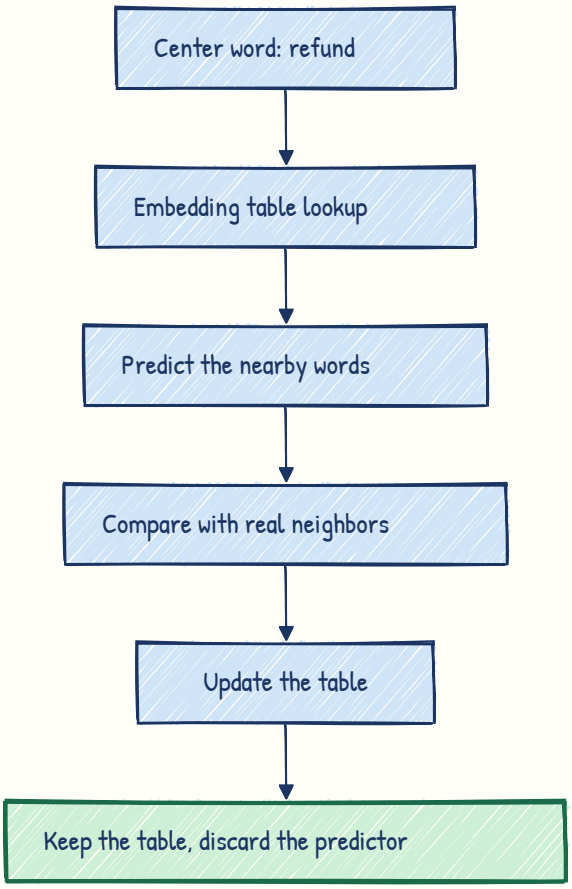


Figure 1. Skip-gram word2vec. The prediction task is only an excuse. The valuable output is the table of word vectors left behind once training ends.

The paper proposed two architectures, **CBOW** and **skip-gram**, and reported training on a 1.6-billion-word corpus in under a day. To avoid computing over the entire vocabulary at each step, later work used **negative sampling**: tell real neighbors apart from a few random words.

An **embedding** simply means a learned, dense vector like these. Your transformer's first layer is an embedding table, trained in much the same spirit (AI02 L2).

Representation	Size	Similar words nearby?	Limits
One-hot or bag of words	Vocabulary size, mostly zeros	No	No meaning
TF-IDF	Vocabulary size, mostly zeros	No	Better weights, same gaps
PPMI context	Vocabulary size	Yes	Large
word2vec	Usually hundreds of numbers	Yes	One vector per word

What's still broken

word2vec gives each word **one vector**. So "charge" has the same vector in "charge my card" and "charge the battery", a muddled average of meanings. It also ignores order: "not approved" and "approved" become almost the same when averaged. To solve these, the vector has to depend on the surrounding sentence, and that requires sequence modeling.

Stage 3: Modeling sequences

Recurrent neural networks

An **RNN** reads one token at a time and maintains a **hidden state**, a memory vector updated at each step.

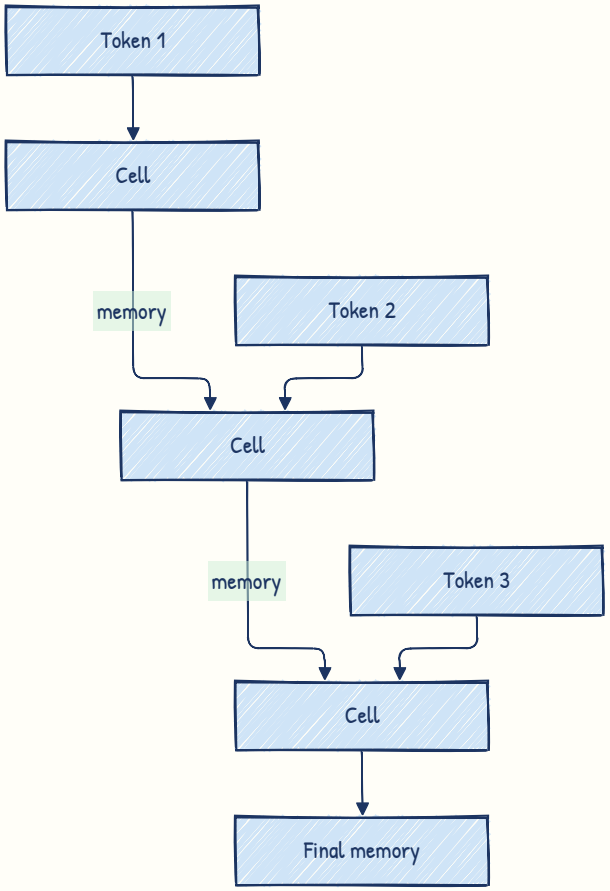


Figure 2. An RNN applies the same cell at every step, passing its memory forward. Reading must proceed in order, so it can't be parallelized across positions.

It's elegant, and it has two serious problems.

Problem 1: it forgets. Training multiplies through each step, so the gradient problem from AI01 L3 comes back, across time rather than layers. As an illustration, if each step keeps 90% of a signal:

```
signal after  5 RNN steps at 0.9 per step = 0.59049
signal after 20 RNN steps at 0.9 per step = 0.12158
signal after 50 RNN steps at 0.9 per step = 0.00515
```

Problem 2: it's sequential. Step 50 can't begin until step 49 is finished, so GPUs can't process positions in parallel.

LSTM and GRU: better memory

The **LSTM** (Hochreiter and Schmidhuber, 1997) added a separate memory line called the **cell state**, controlled by learned **gates**. The **GRU** is a simpler variant with fewer gates.

Gate	Question it answers	Support-ticket example
Forget gate	What should be dropped from memory?	A new topic started, so forget the old one
Input gate	What new information should be stored?	"Order number 12345" is worth keeping
Output gate	What should be passed on now?	The next word needs to know the subject

Because memory is carried mostly by additions rather than repeated multiplications, signal survives much longer. LSTMs were the leading approach to language tasks for years. Problem 2 remained, however, and so did a third problem, which turned out to be decisive.

Sequence-to-sequence, and the bottleneck

For translation or summarization, an **encoder** RNN reads the input into its final memory, and a **decoder** RNN writes the output from it.

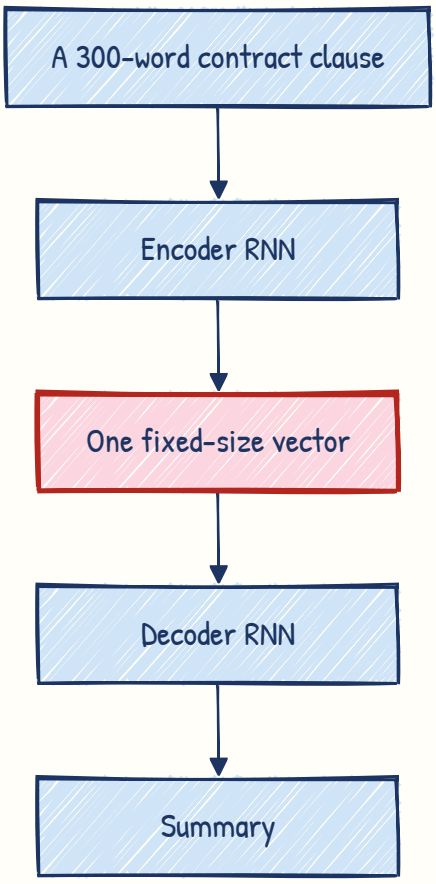


Figure 3. The bottleneck. A long document must pass through one fixed-size vector, whatever its length.

Imagine reading a contract, closing it, summarizing the whole thing on a sticky note, and then answering questions using only that note. Bahdanau, Cho and Bengio described the exact limit in their 2014 paper: "the use of a fixed-length vector is a bottleneck". Their solution was to let the decoder look back at every input word, focusing on the relevant ones at each step. That is **attention**.

> *Key idea*
>
> **Attention began as a patch on RNNs, and it later turned out the RNN could be removed altogether.** That is the 2017 paper "Attention Is All You Need", and it's where AI02 L2 and L3 pick up.

Autoencoders, briefly

An **autoencoder** is trained to compress an input into a small vector and then rebuild it. Since the target is the input, it needs no labels. Uses include dimensionality reduction and anomaly detection: something unusual reconstructs poorly, so high reconstruction error can flag an unusual transaction. It's also a stepping stone to generative models (AI02 L7).

How the field solved it

Problem	Solution	Remaining weakness
Text isn't numeric	Bag of words	No meaning or order
Common words dominate	TF-IDF, then BM25	Synonyms missed
Synonyms look unrelated	Distributional vectors, word2vec	One vector per word
Order is lost	RNN	Forgets, and is sequential
RNNs forget	LSTM, GRU	Still sequential, still a bottleneck
Fixed-size bottleneck	Attention	Still attached to an RNN
Sequential processing	Transformer (AI02 L3)	Cost grows with context length

Anthropic connection: none of these was Anthropic's invention. One practical note carries forward: Anthropic doesn't offer its own embedding model, and its guide covers Voyage AI, while advising you to assess multiple vendors. So a RAG system built on Claude normally uses a separate embedding provider. AI04 L1 explains how to choose.

The production architecture

This is a recommended reference design, not a specific company's system. It evolves a ticket-routing service, and shows that the older methods still have a role.

Baseline: keyword rules. Fast, cheap and blind to synonyms.

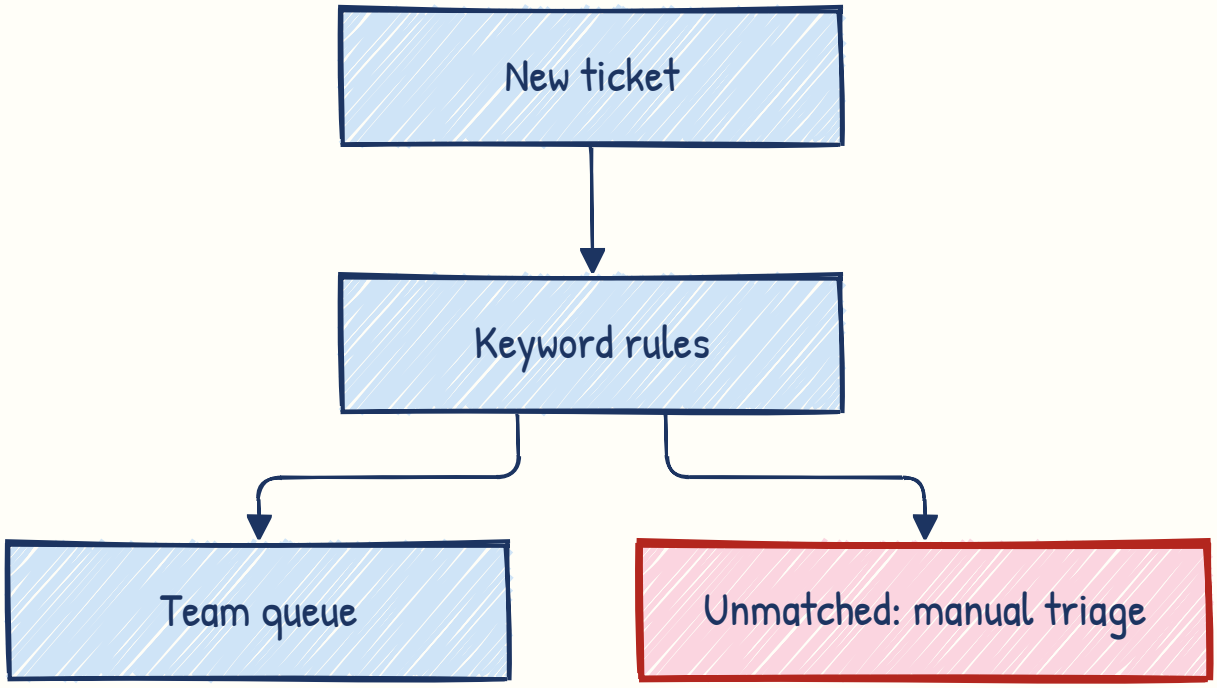


Figure 4. Baseline. Keyword rules are cheap and explainable, but miss any wording that wasn't anticipated.

Improved: a classifier over TF-IDF. It learns from history and works well on common categories. New problem: **it still can't handle wording absent from its training data, and every new category needs fresh labels.**

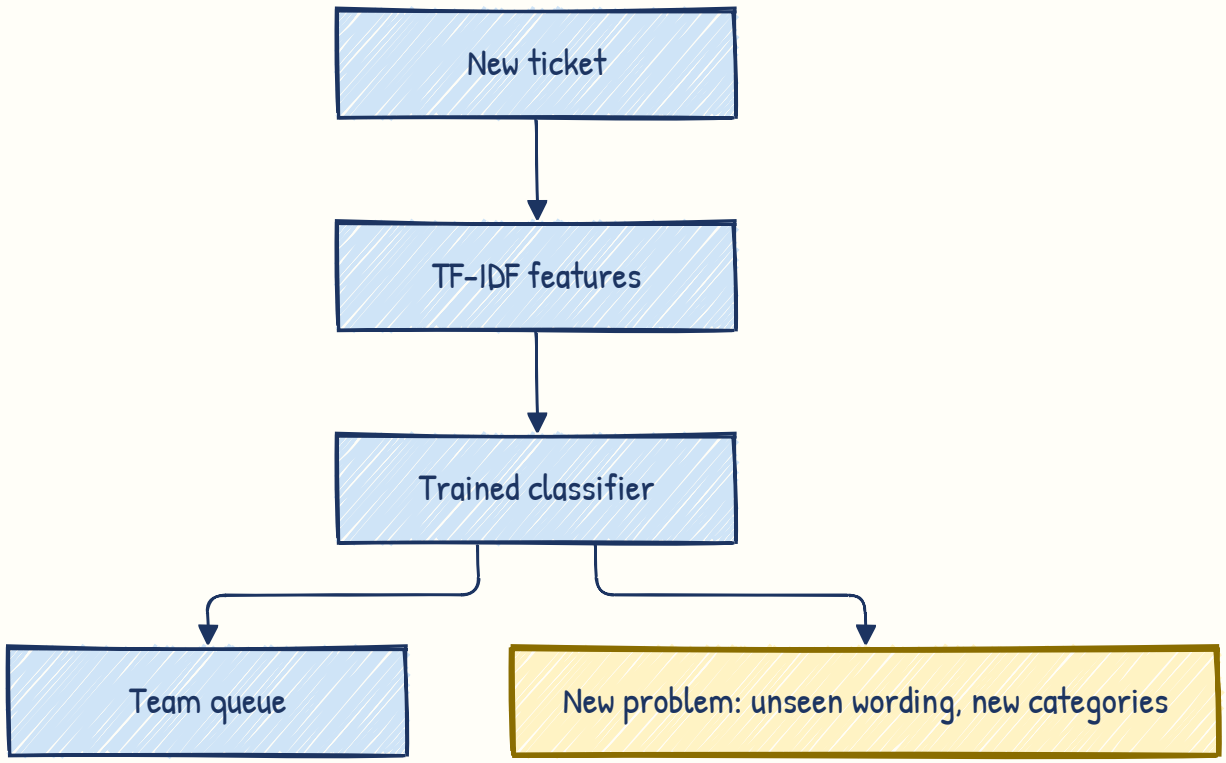


Figure 5. Improved. A trained classifier learns from history, but needs labels for each category and stays weak on unseen wording.

Production: tiered by cost and confidence. Cheap methods handle what they can, and costly ones take the rest.

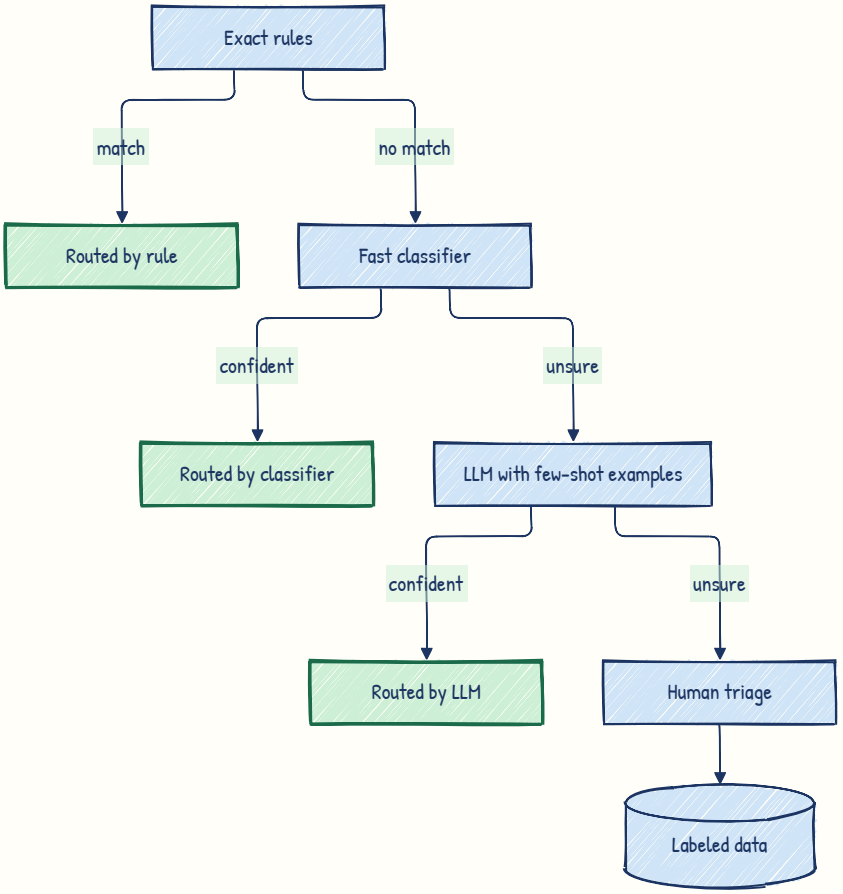


Figure 6. Production. Each tier routes what it's confident about, and passes only the rest down. LLM calls are reserved for the hard remainder, and human decisions become training data.

Component	Why it exists	How it communicates
Exact rules	IDs and codes need no intelligence	In-process, in microseconds
Fast classifier	Handles common cases cheaply	In-process, or a small service
LLM tier	Handles unusual wording, with no retraining	Async API call, with a tight budget (AI03)
Human triage	Owns the remaining uncertainty	Work queue
Labeled data	Each human decision improves the cheap tiers	Written back to the training dataset

Failures

Failure	Containment
LLM tier unavailable	Keep the classifier's best guess and flag it, or send to humans
Classifier confidence unreliable	Calibrate using a labeled sample (AI10 L3)
New product name appears	LLM handles it first, and retraining catches up later
Prompt injection in ticket text	Treat ticket text as untrusted data (AI09 L2)

Trade-offs

Decision	You gain	It costs
Keyword or TF-IDF	Cheap, fast, explainable, exact terms	Misses meaning
Embeddings	Finds meaning	Provider, storage, weak on exact codes
LLM	Understands context, no training	Latency and cost per call
Tiered approach	Best cost for each case	Several components to run

When not to use embeddings or LLMs: exact lookups, small closed vocabularies, or anywhere keyword search already passes your evaluation.

Common mistakes

1. **Thinking word vectors understand words.** They capture co-occurrence statistics, and that can include biases in the training text.
2. **Averaging word vectors and expecting order to survive.** "Not approved" and "approved, not" become the same.
3. **Dropping keyword search once embeddings arrive.** It remains better for IDs and codes.
4. **Comparing vectors from different models.** They aren't in the same space (AI04 L1).
5. **Skipping a TF-IDF baseline.** If it meets the requirement, you've saved real money.

What to say in an interview

For "why did transformers replace RNNs?", give the chain of problems: "RNNs forget across long sequences, they process one step at a time, which blocks parallelism, and seq2seq forces everything through one fixed-size vector. Attention removed the bottleneck by letting the decoder see all inputs, and transformers then dropped the recurrence." Telling it as a chain shows you understand why, not just what.

Certification lens

How this lesson maps to CCAR-F

Not tested. Embedding models and tokenization specifics are explicitly out of scope. This lesson exists for AI02, AI04 and interviews.

These are original practice questions for understanding, not exam simulations.

Q Q1 · Match the following · Easy. Match each method to its weakness. Methods: (1) TF-IDF, (2) word2vec, (3) LSTM. Weaknesses: (a) processes tokens strictly in sequence, (b) treats synonyms as unrelated, (c) one vector per word, whatever its context.

A 1-b, 2-c, 3-a.

Q Q2 · Calculation · Medium. A term appears in 2 of 200 documents. Another appears in 100 of 200. Compute each IDF using the natural log. What's the ratio?

A `ln(100)` = 4.605 and `ln(2)` = 0.693, so the rarer term is weighted about 6.6 times as heavily.

Q Q3 · Best decision · Hard. Support search must find documents by meaning, and also by exact error codes like `ERR-4012`. Which approach fits best? (A) Replace keyword search with vectors. (B) Keep keywords only and add a synonym list. (C) Run both and merge the results. (D) Put all documents in every prompt.

A C. Each method covers the other's weakness: vectors handle meaning, and keywords handle exact codes (AI04 L3). A loses exact matching. B needs every synonym anticipated in advance. D is only workable for small collections.

Revision - Lesson 4

→ Bag of words counts terms and loses meaning and order. Synonyms share nothing.

→ TF-IDF weights rare words more. BM25 refines it and remains in production use.

→ Distributional hypothesis: words in similar contexts have similar meanings.

→ word2vec learns short, dense vectors through a prediction task. The vectors are the product.

→ One vector per word can't separate the meanings of "charge".

→ RNNs carry a memory, but they forget and can't run positions in parallel.

→ LSTMs and GRUs use gates to remember longer.

→ Seq2seq squeezes input through one vector. Attention was invented to remove that bottleneck.

→ Autoencoders compress and rebuild, with no labels. High rebuild error can flag anomalies.

→ Anthropic offers no embedding model of its own, so RAG normally uses another provider.

Your notes

Module Revision

AI and ML fundamentals - on one page

The one sentence · ML infers behavior from examples, so it handles what you can't specify, is sometimes wrong, and decays as the world changes.

The training loop · Forward pass, measure the loss, backpropagate, update. A threshold search and an LLM both follow this shape.

The assumption · The future resembles the training data. Drift breaks that silently, so monitor.

Who owns what · Code owns guarantees, models own patterns, and people own uncertainty.

Measuring · Never accuracy alone on rare events. Use precision and recall on held-out data, derive thresholds from costs, and check every segment.

Too good to be true · A near-perfect score suggests leakage, so check each feature was known at decision time.

Algorithms · For tables, start with logistic regression, then gradient boosting. Use deep learning mainly for language and images.

Neural networks · Non-linear layers learn interactions. Backpropagation makes gradients affordable. ReLU, normalization and residual connections keep deep networks trainable.

Language · Counts, then context vectors, then sequences, then attention. Each stage solved a real weakness in the last. BM25 remains useful for exact terms.

Self-assessment

Answer without looking back. Each has a specific answer in this module.

#	Question
1	What's the difference between AI, ML, deep learning and an LLM?
2	What's the difference between a parameter and a hyperparameter?
3	What assumption does generalization rely on?
4	Name the three types of drift, with an example of each.
5	How does a model's failure differ from a service's failure?
6	Which rule belongs in code rather than a model, and why?
7	Why did "never fraud" score 97.9%?
8	Define precision and recall. Which do you favor for automated PR comments?
9	Review costs $5, and a miss costs $200. What threshold does this suggest?
10	Give three examples of data leakage.
11	What do the train, validation and test sets each do?
12	What's the difference between bagging and boosting?
13	Why can an aggregate score hide a broken segment?
14	Describe the release ladder after an offline pass.
15	Why can't logistic regression learn XOR?
16	Why does a network need non-linear activations?
17	What does cross-entropy penalize most?
18	Why is backpropagation so much cheaper than nudging each weight?
19	What does a `NaN` loss most often indicate?
20	What does Adam combine?
21	What causes vanishing gradients, and name three fixes.
22	What does a residual connection do?
23	Why do synonyms have zero similarity under bag of words?

#	Question
24	What's the distributional hypothesis?
25	What does word2vec keep when training ends?
26	Why can't one vector per word handle "charge"?
27	Give RNNs' three weaknesses.
28	What problem was attention originally invented to solve?
29	Does Anthropic provide an embedding model? What follows from that?

Where this module is used

Idea	Where it returns
Code owns guarantees	AI06 L8 hooks, AI09, TS 1.4 and 1.5
Precision and recall	AI05 L6, AI10 L3, AI12 L5, TS 4.1
Segment analysis	AI10 L3, TS 5.5
Overfitting and held-out data	AI10 L3 (prompts can overfit evals too)
Drift and monitoring	AI10 L4
Cross-entropy, backpropagation, Adam	AI02 L4, AI11 L1
Residual connections, normalization	AI02 L3
Softmax	AI02 L5
Word vectors and the bottleneck	AI02 L2, AI04 L1
TF-IDF and BM25	AI04 L3

Working space